



Chapter 11

Data Link Control

11-1 FRAMING

*The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.*

Topics discussed in this section:

Fixed-Size Framing

Variable-Size Framing

Figure 11.1 *A frame in a character-oriented protocol*

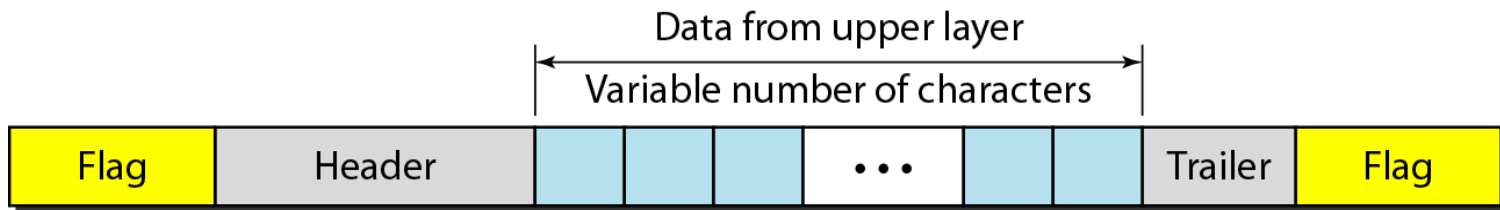
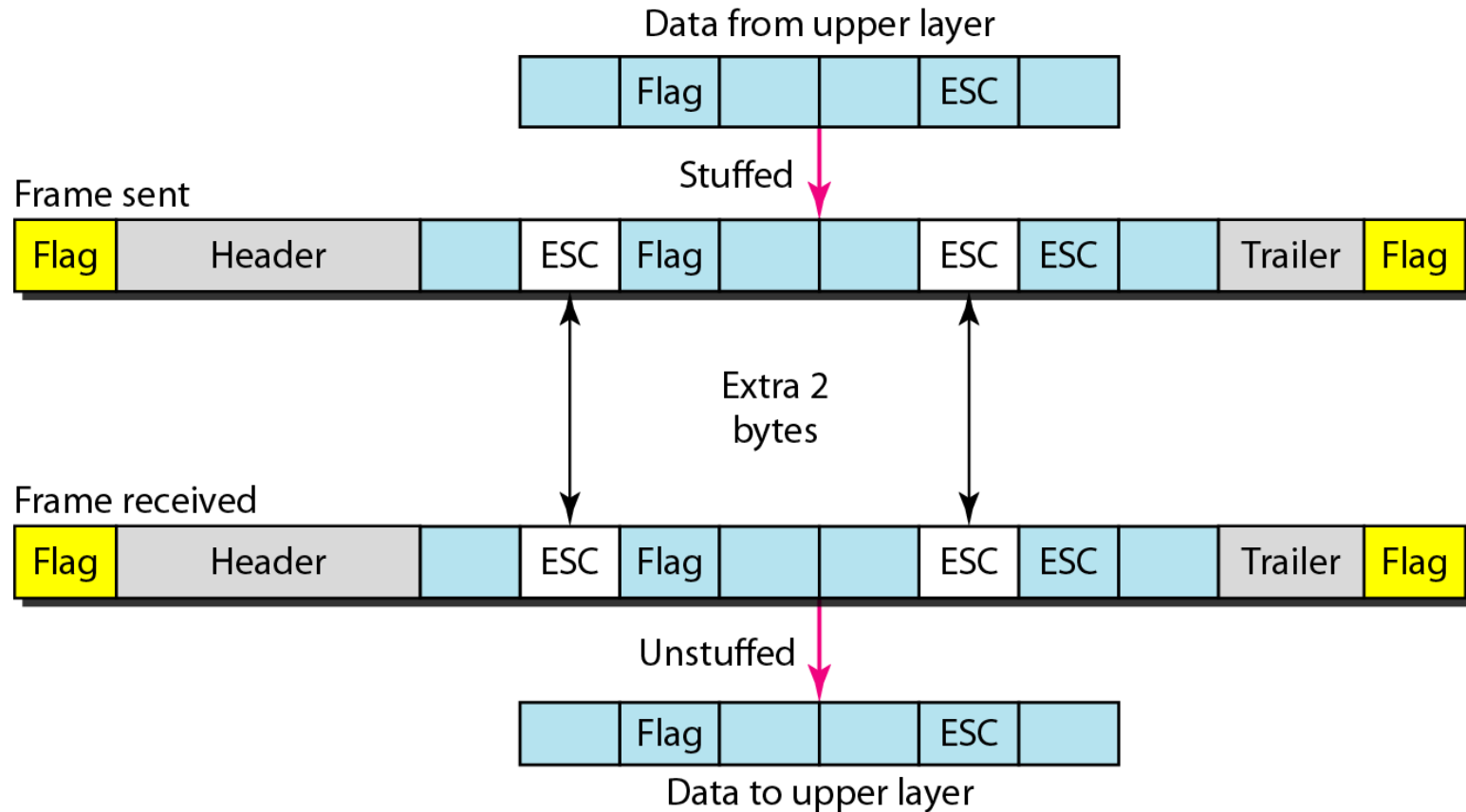


Figure 11.2 *Byte stuffing and unstuffing*

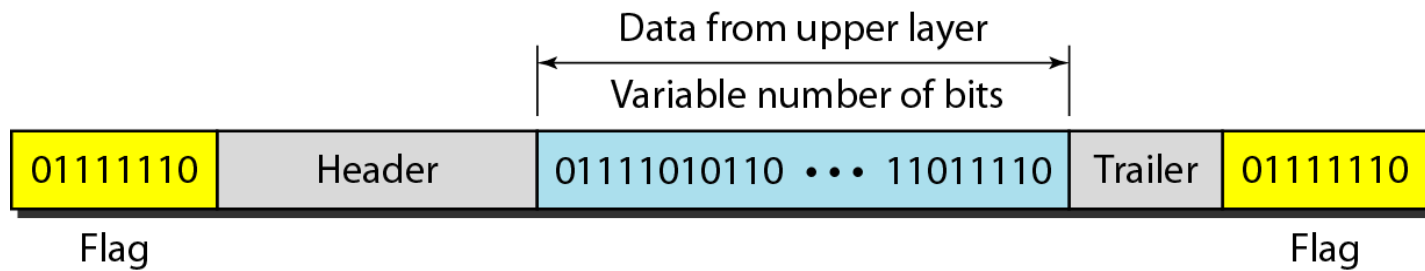




Note

Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text.

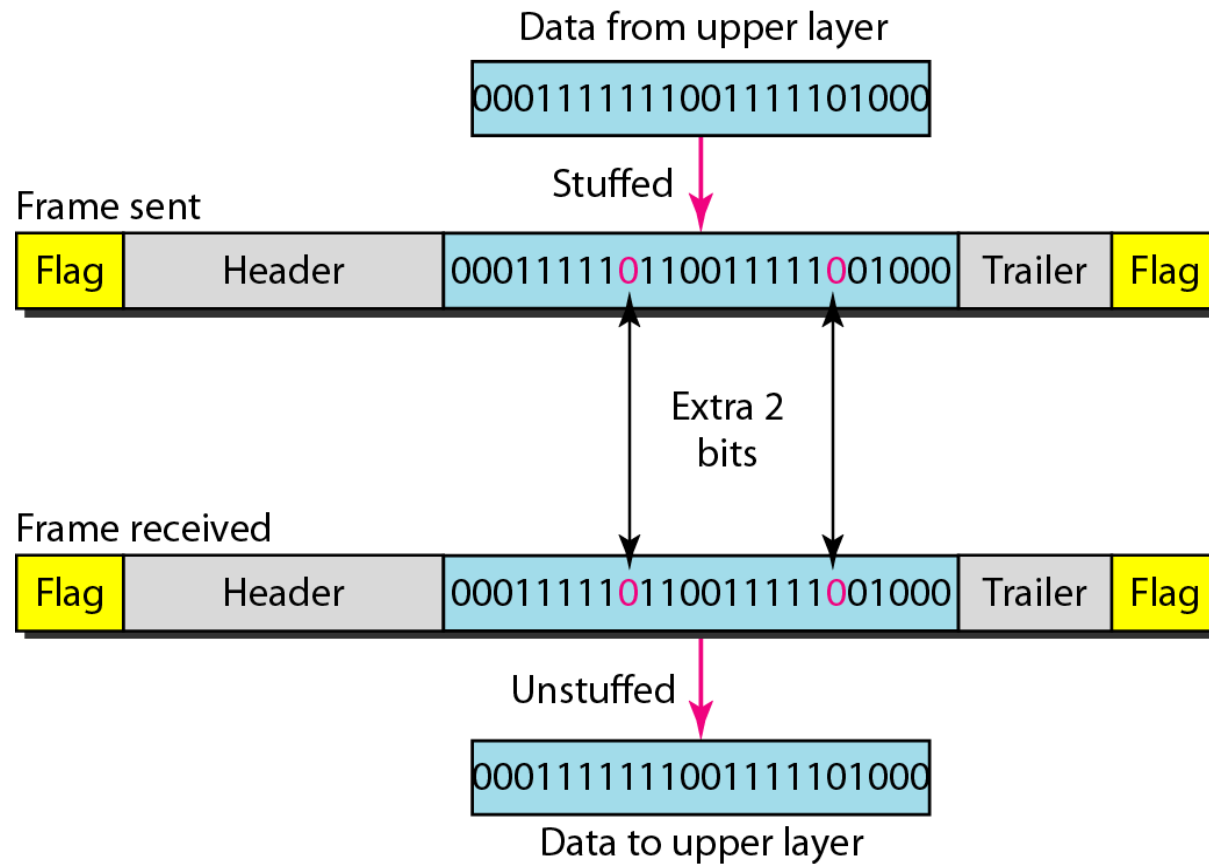
Figure 11.3 *A frame in a bit-oriented protocol*



Note

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

Figure 11.4 *Bit stuffing and unstuffing*



11-2 FLOW AND ERROR CONTROL

*The most important responsibilities of the data link layer are **flow control** and **error control**. Collectively, these functions are known as **data link control**.*

Topics discussed in this section:

Flow Control

Error Control



Note

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.



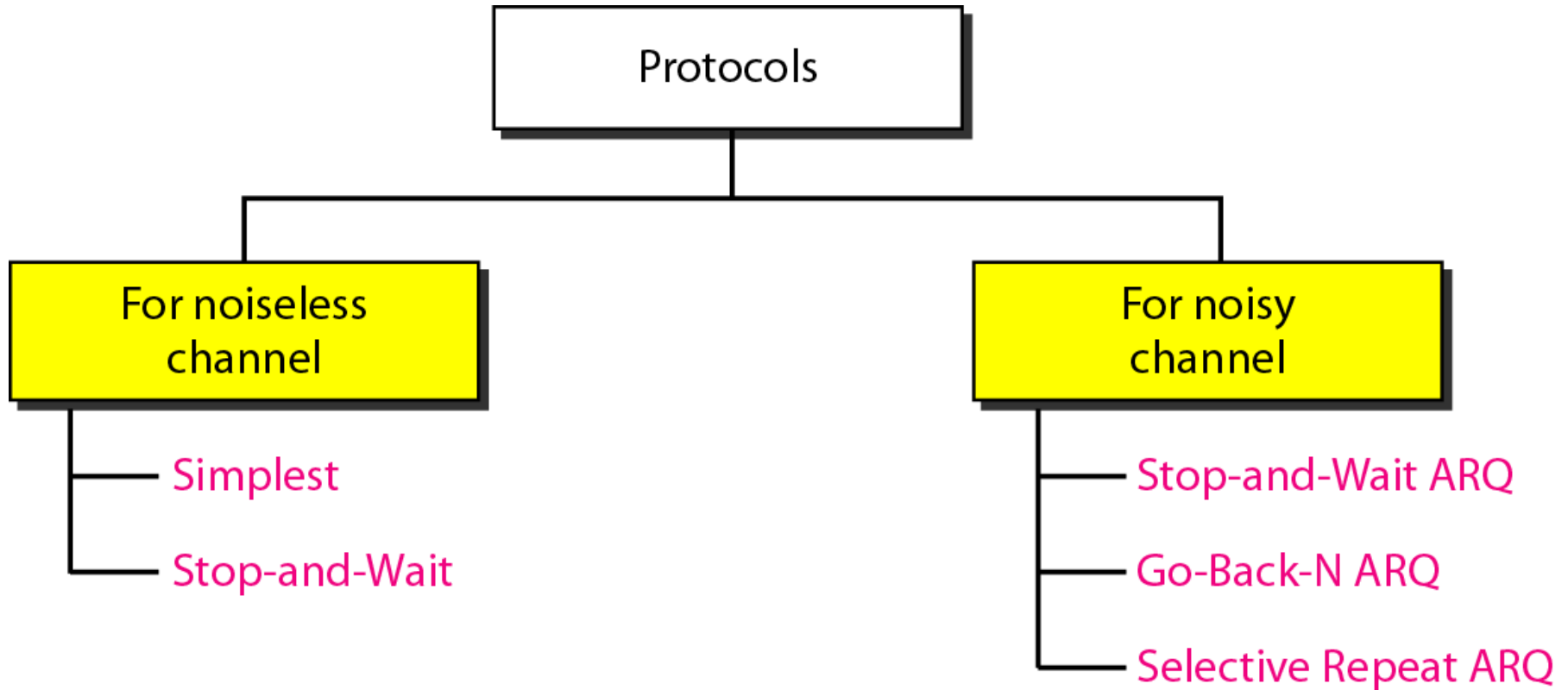
Note

Error control in the data link layer is based on automatic repeat request, which is the retransmission of data.

11-3 PROTOCOLS

Now let us see how the data link layer can combine framing, flow control, and error control to achieve the delivery of data from one node to another. The protocols are normally implemented in software by using one of the common programming languages. To make our discussions language-free, we have written in pseudocode a version of each protocol that concentrates mostly on the procedure instead of delving into the details of language rules.

Figure 11.5 *Taxonomy of protocols discussed in this chapter*



11-4 NOISELESS CHANNELS

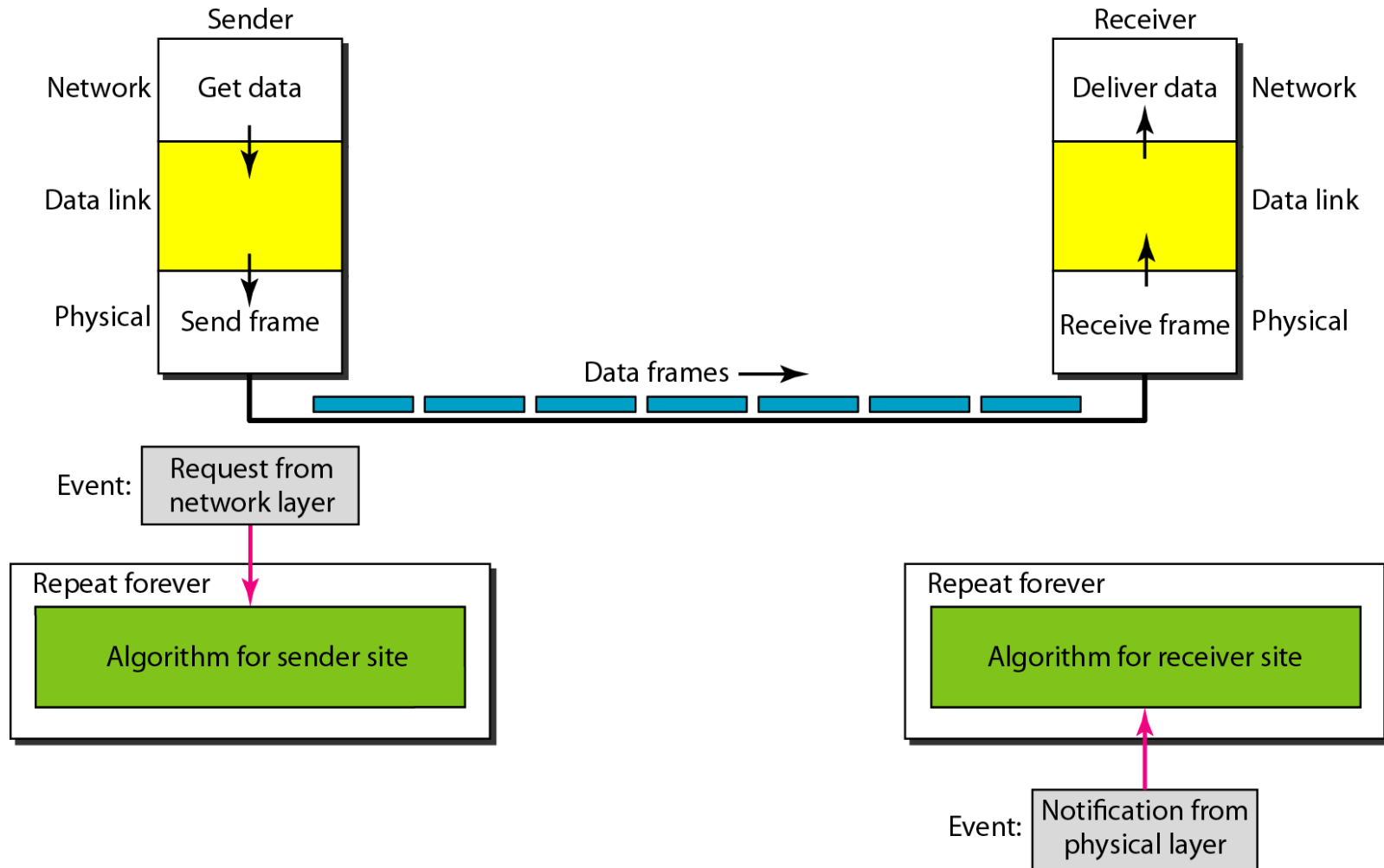
Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel.

Topics discussed in this section:

Simplest Protocol

Stop-and-Wait Protocol

Figure 11.6 *The design of the simplest protocol with no flow or error control*



Algorithm 11.1 *Sender-site algorithm for the simplest protocol*

```
1 while(true)                                // Repeat forever
2 {
3     WaitForEvent();                          // Sleep until an event occurs
4     if(Event(RequestToSend))                //There is a packet to send
5     {
6         GetData();
7         MakeFrame();
8         SendFrame();                          //Send the frame
9     }
10 }
```

Algorithm 11.2 *Receiver-site algorithm for the simplest protocol*

```
1 while(true)                                // Repeat forever
2 {
3     WaitForEvent();                          // Sleep until an event occurs
4     if(Event(ArrivalNotification))          //Data frame arrived
5     {
6         ReceiveFrame();
7         ExtractData();
8         DeliverData();                       //Deliver data to network layer
9     }
10 }
```



Example 11.1

Figure 11.7 shows an example of communication using this protocol. It is very simple. The sender sends a sequence of frames without even thinking about the receiver. To send three frames, three events occur at the sender site and three events at the receiver site. Note that the data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.

Figure 11.7 *Flow diagram for Example 11.1*

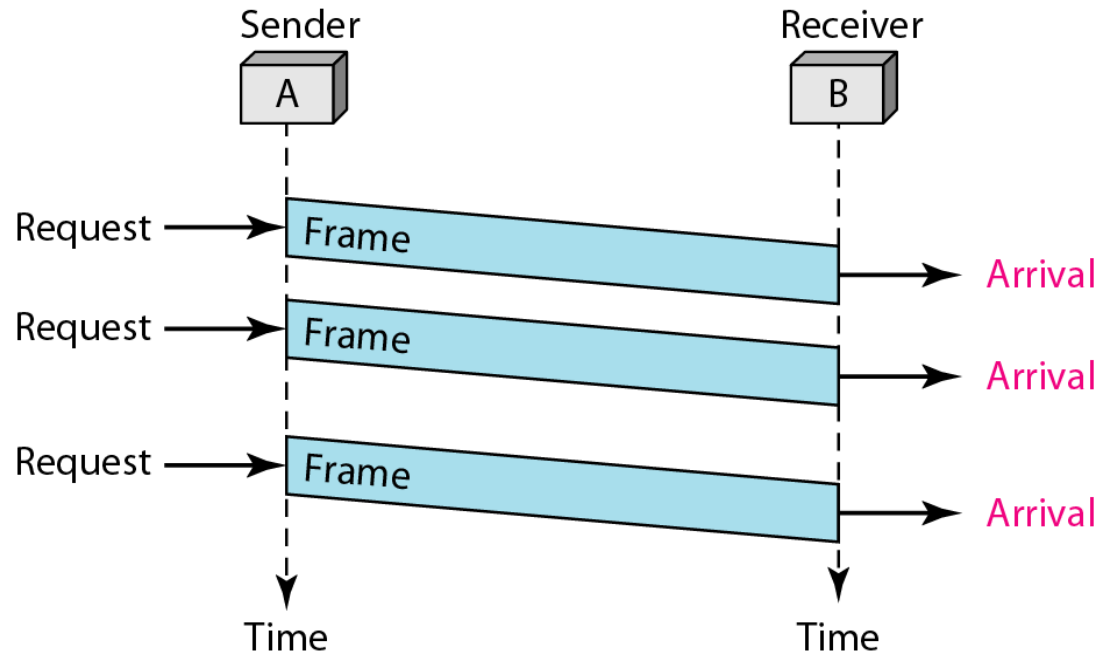
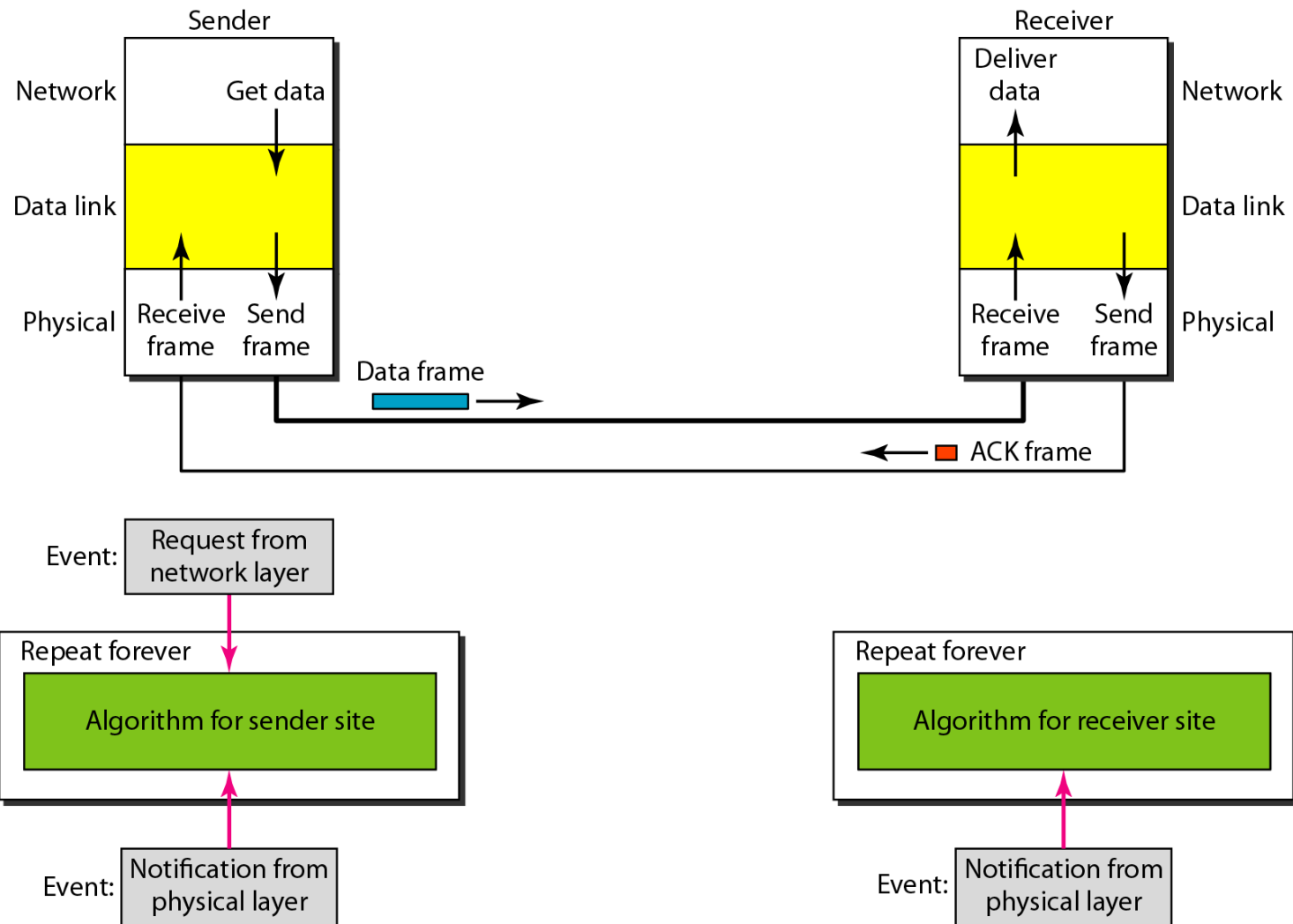


Figure 11.8 *Design of Stop-and-Wait Protocol*



Algorithm 11.3 *Sender-site algorithm for Stop-and-Wait Protocol*

```
1 while(true)                                //Repeat forever
2   canSend = true                            //Allow the first frame to go
3   {
4     WaitForEvent();                        // Sleep until an event occurs
5     if(Event(RequestToSend) AND canSend)
6     {
7       GetData();
8       MakeFrame();
9       SendFrame();                        //Send the data frame
10      canSend = false;                    //Cannot send until ACK arrives
11    }
12    WaitForEvent();                        // Sleep until an event occurs
13    if(Event(ArrivalNotification) // An ACK has arrived
14    {
15      ReceiveFrame();                      //Receive the ACK frame
16      canSend = true;
17    }
18 }
```

Algorithm 11.4 Receiver-site algorithm for Stop-and-Wait Protocol

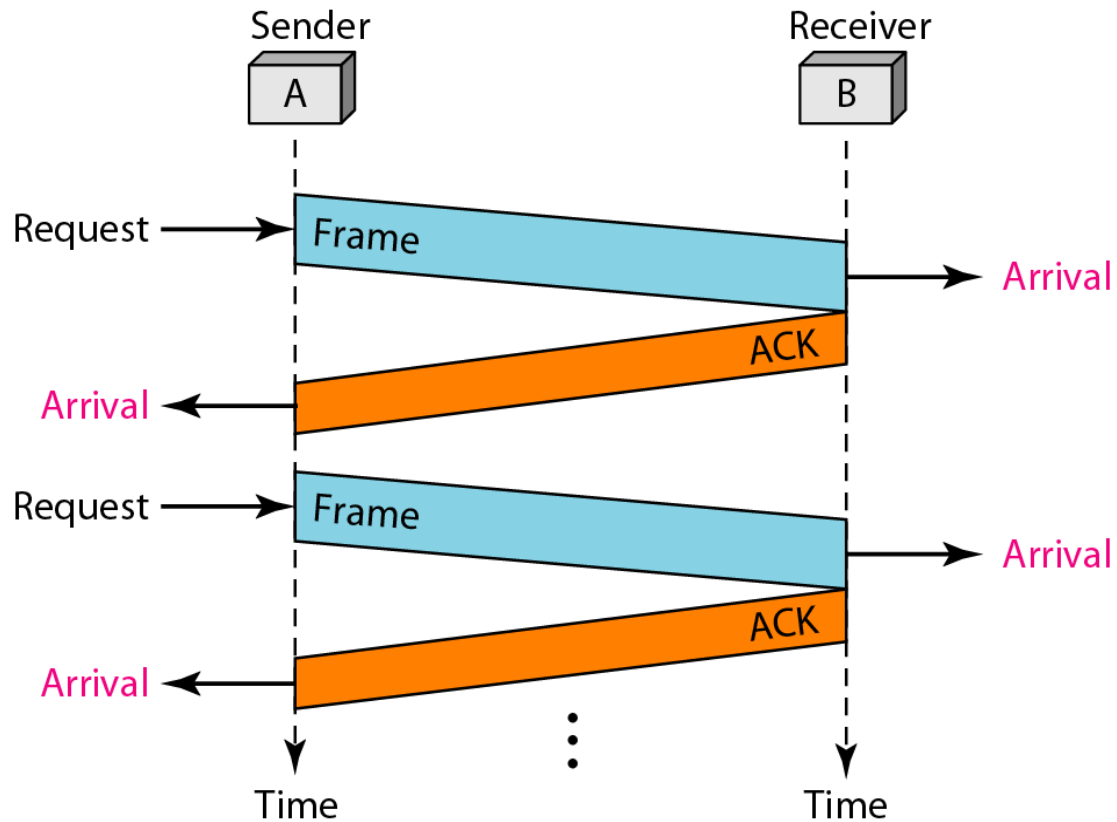
```
1 while(true)                                //Repeat forever
2 {
3     WaitForEvent();                          // Sleep until an event occurs
4     if(Event(ArrivalNotification))           //Data frame arrives
5     {
6         ReceiveFrame();
7         ExtractData();
8         Deliver(data);                       //Deliver data to network layer
9         SendFrame();                          //Send an ACK frame
10    }
11 }
```



Example 11.2

Figure 11.9 shows an example of communication using this protocol. It is still very simple. The sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame. Note that sending two frames in the protocol involves the sender in four events and the receiver in two events.

Figure 11.9 *Flow diagram for Example 11.2*



11-5 NOISY CHANNELS

Although the Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent. We discuss three protocols in this section that use error control.

Topics discussed in this section:

Stop-and-Wait Automatic Repeat Request

Go-Back-N Automatic Repeat Request

Selective Repeat Automatic Repeat Request

Note

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

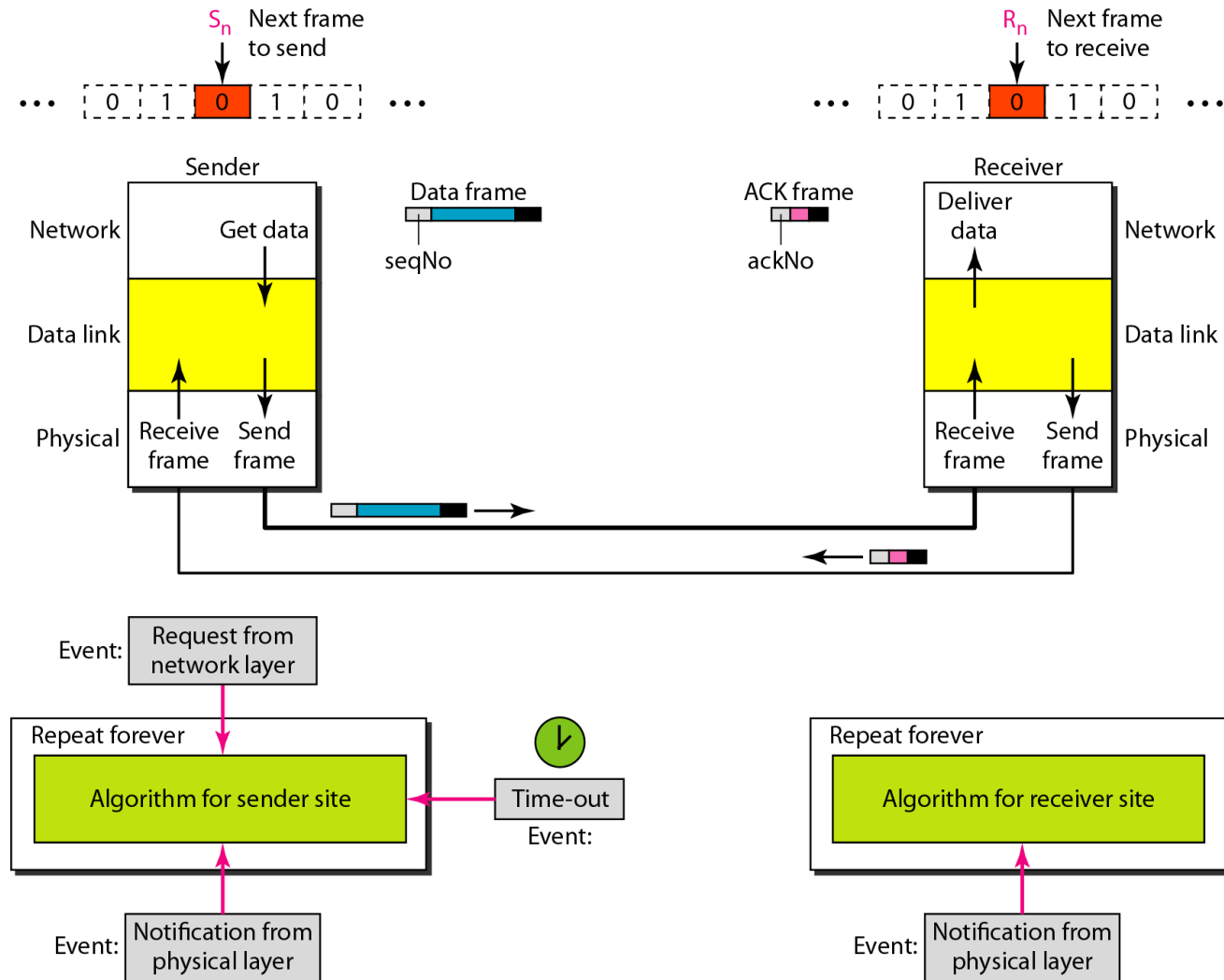
Note

**In Stop-and-Wait ARQ, we use sequence numbers to number the frames.
The sequence numbers are based on modulo-2 arithmetic.**

Note

In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

Figure 11.10 *Design of the Stop-and-Wait ARQ Protocol*



Stop-and-Wait ARQ

- *It is the simplest flow and error control mechanism . A transmitter sends a frame then stops and waits for an acknowledgment.*
- *Stop-and-Wait ARQ has the following features:*
 - ✓ *The sending device keeps a copy of the sent frame transmitted until it receives an acknowledgment(ACK)*
 - ✓ *The sender starts a timer when it sends a frame. If an ACK is not received within an allocated time period, the sender resends it*
 - ✓ *Both frames and acknowledgment (ACK) are numbered alternately 0 and 1(two sequence number only)*
 - ✓ *This numbering allows for identification of frames in case of duplicate transmission*

Stop-and-Wait ARQ

- *The acknowledgment number defines the number of next expected frame. (frame 0 received ACK 1 is sent)*
- *A damage or lost frame treated by the same manner by the receiver*
- *If the receiver detects an error in the received frame, or receives a frame out of order it simply discards the frame*
- *The receiver send only positive ACK for frames received safe; it is silent about the frames damage or lost.*
- *The sender has a control variable S that holds the number of most recently sent frame (0 or 1). The receiver has control variable R , that holds the number of the next frame expected (0 or 1)*

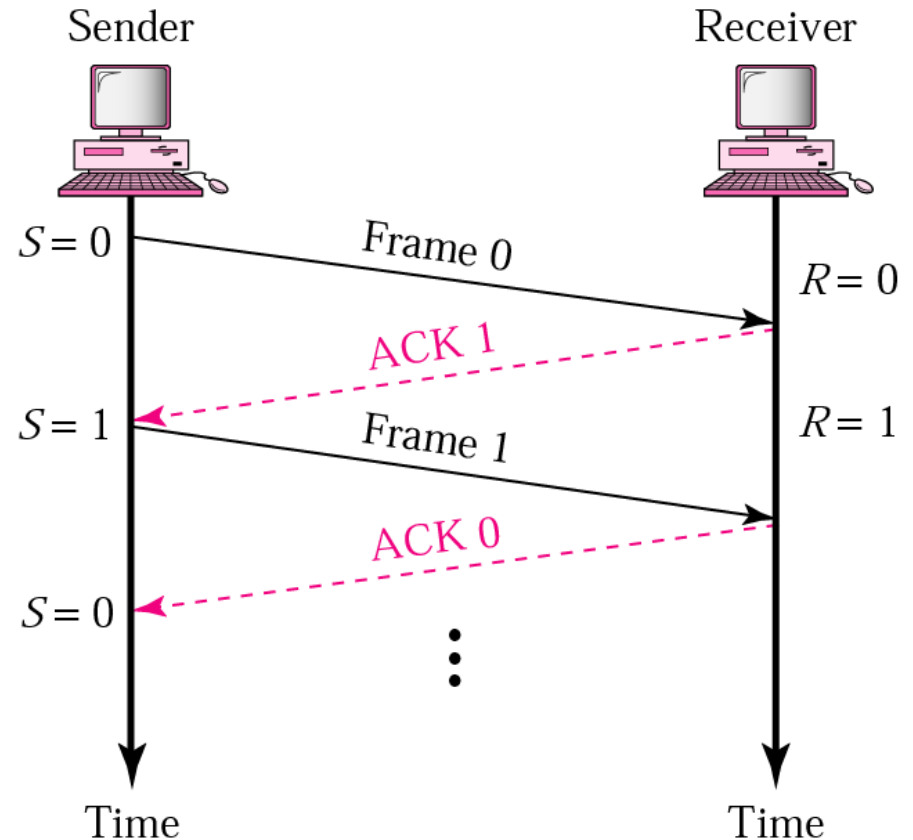
Cases of Operations:

- 1. Normal operation***
- 2. The frame is lost***
- 3. The Acknowledgment (ACK) is lost***
- 4. The Ack is delayed***

Stop-and-Wait ARQ

Normal operation

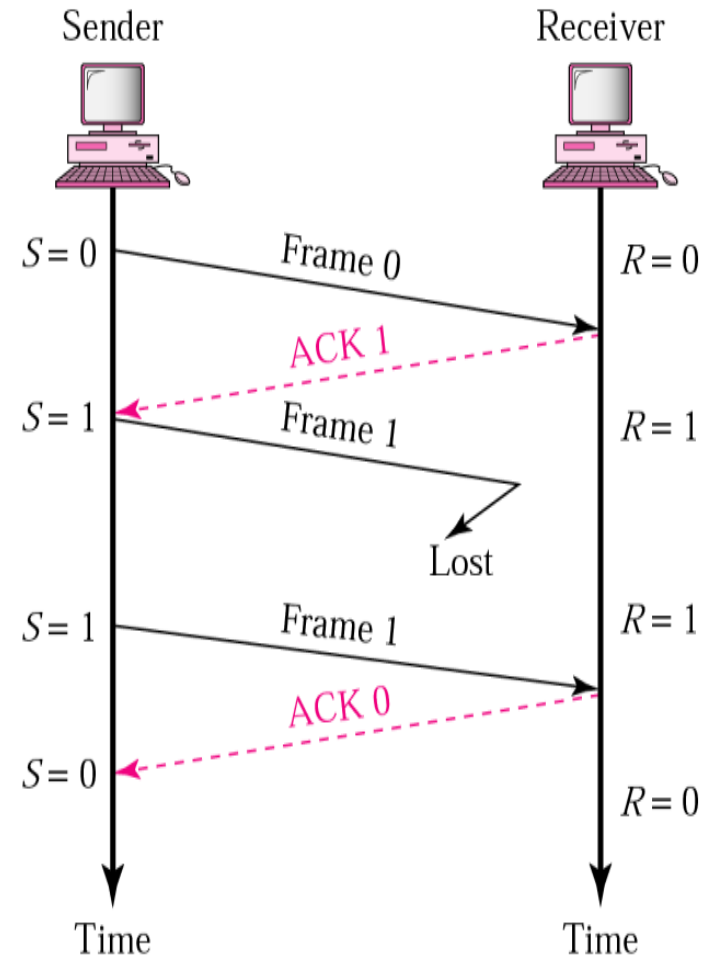
- *The sender will not send the next frame until it is sure that the current one is correctly receive*
- *sequence number is necessary to check for duplicated frames*



1. Stop and Wait ARQ

2. Lost or damaged frame

- *A damage or lost frame treated by the same manner by the receiver.*
- *No NACK when frame is corrupted / duplicate*

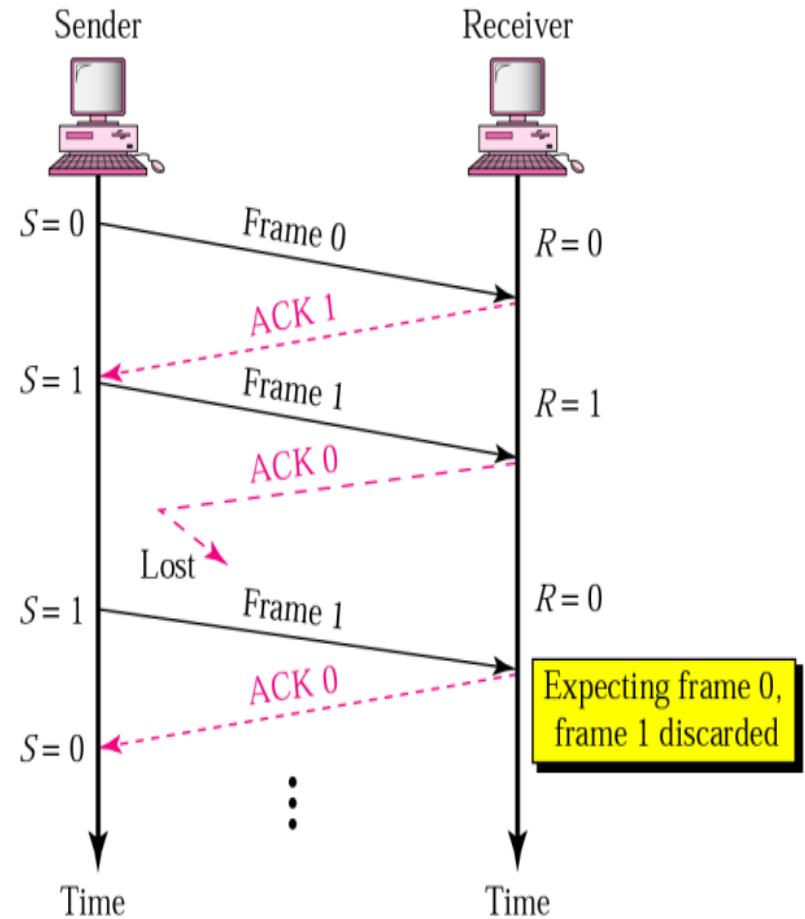
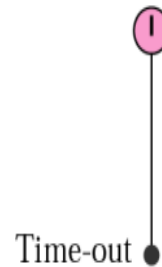


Stop-and-Wait ARQ

3. Lost ACK frame



Importance of frame numbering



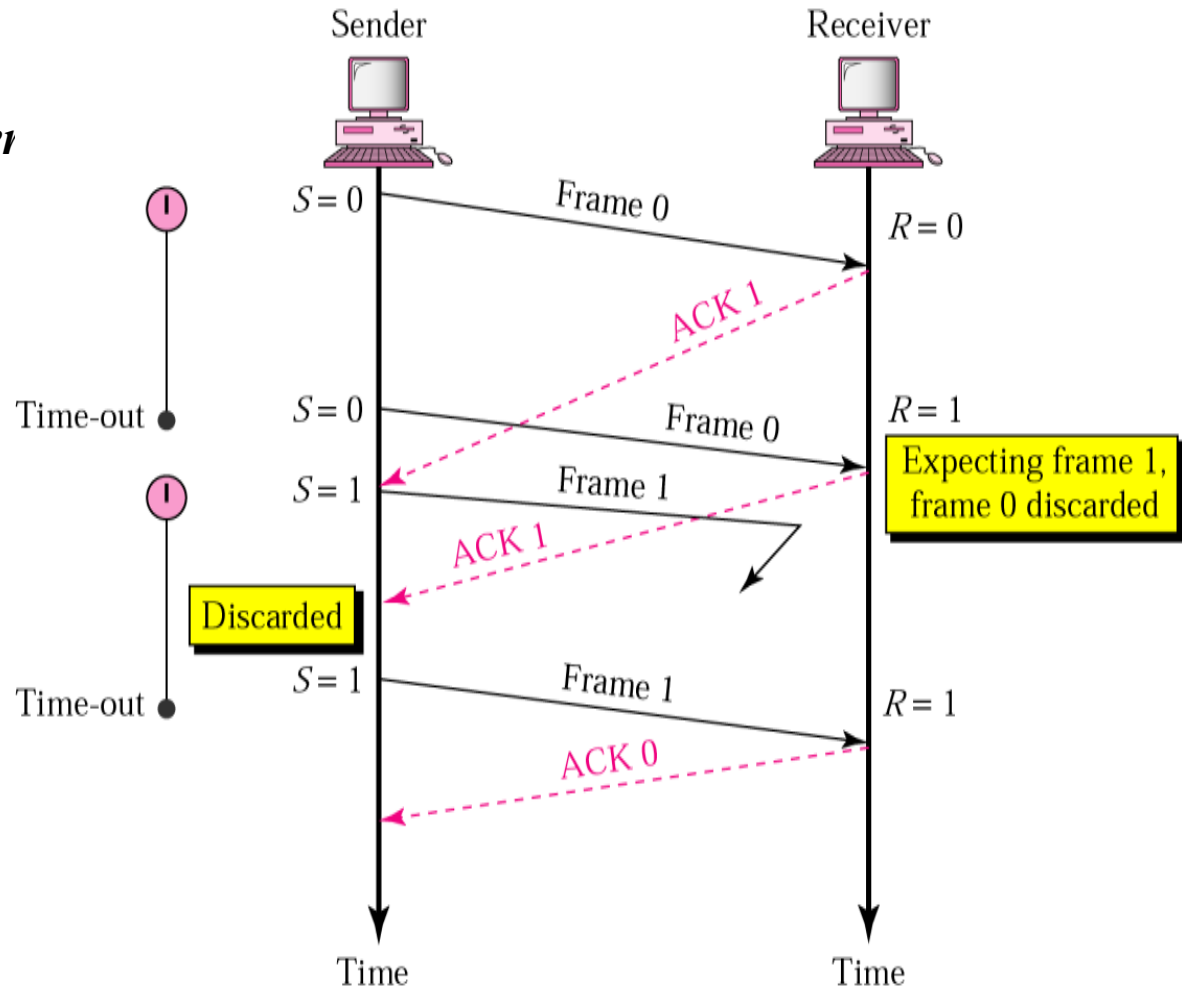
Note

In Stop and-Wait ARQ, numbering frames prevents the retaining of duplicate frames.

Stop-and-Wait ARQ

4. Delayed ACK and lost frame

➤ *Importance of frame number*



Note

Numbered acknowledgments are needed if an acknowledgment is delayed and the next frame is lost.

Algorithm 11.5 *Sender-site algorithm for Stop-and-Wait ARQ*

```
1  Sn = 0;                                // Frame 0 should be sent first
2  canSend = true;                          // Allow the first request to go
3  while(true)                             // Repeat forever
4  {
5      WaitForEvent();                      // Sleep until an event occurs
6      if(Event(RequestToSend) AND canSend)
7      {
8          GetData();
9          MakeFrame(Sn);                  //The seqNo is Sn
10         StoreFrame(Sn);                //Keep copy
11         SendFrame(Sn);
12         StartTimer();
13         Sn = Sn + 1;
14         canSend = false;
15     }
16     WaitForEvent();                      // Sleep
```

(continued)

Algorithm 11.5 *Sender-site algorithm for Stop-and-Wait ARQ* (continued)

```
17     if(Event(ArrivalNotification)           // An ACK has arrived
18     {
19         ReceiveFrame(ackNo);                 //Receive the ACK frame
20         if(not corrupted AND ackNo == Sn) //Valid ACK
21         {
22             Stoptimer();
23             PurgeFrame(Sn-1);                //Copy is not needed
24             canSend = true;
25         }
26     }
27
28     if(Event(TimeOut)                         // The timer expired
29     {
30         StartTimer();
31         ResendFrame(Sn-1);                  //Resend a copy check
32     }
33 }
```

Algorithm 11.6 Receiver-site algorithm for Stop-and-Wait ARQ Protocol

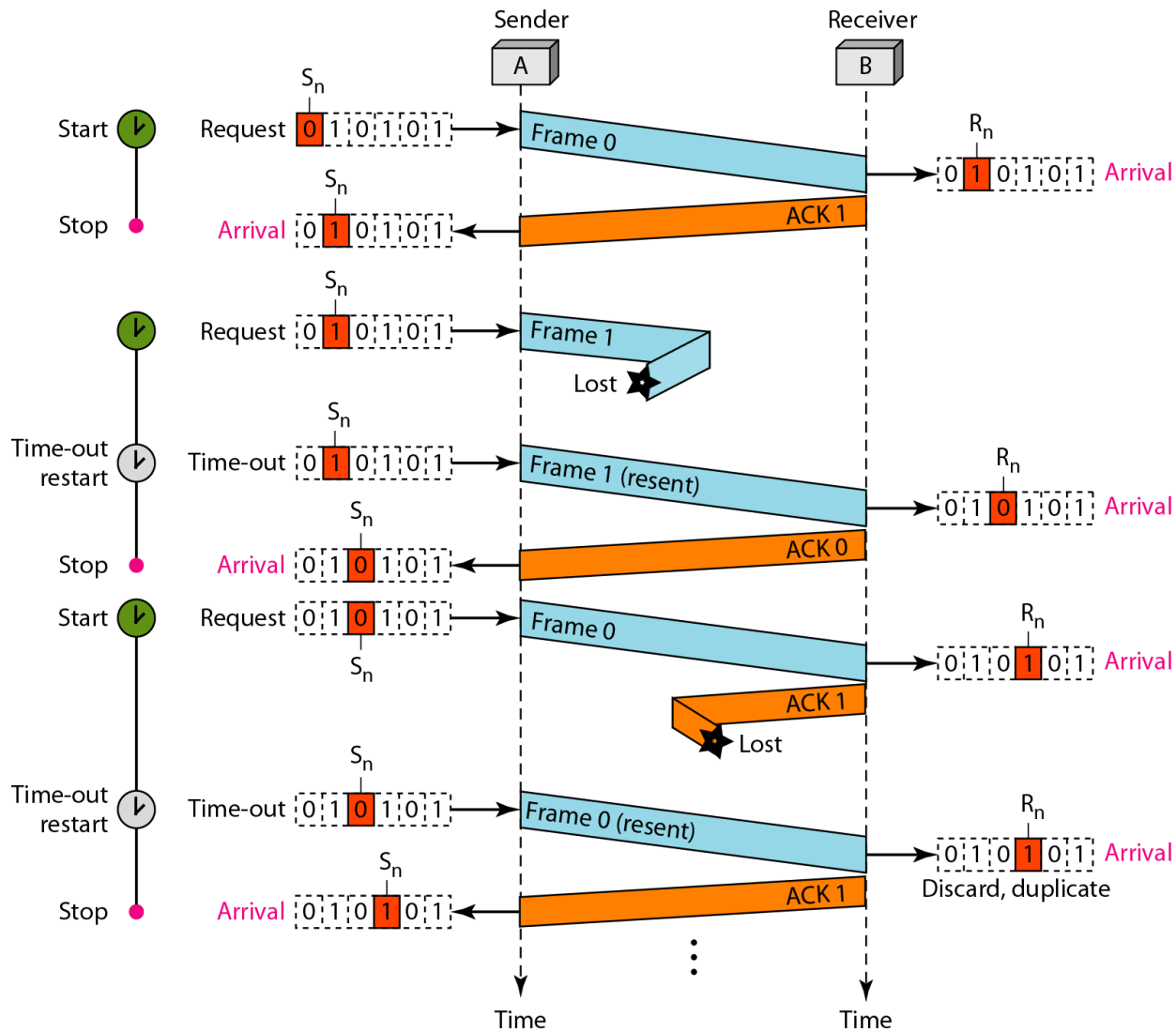
```
1  Rn = 0;                                // Frame 0 expected to arrive first
2  while(true)
3  {
4      WaitForEvent();                      // Sleep until an event occurs
5      if(Event(ArrivalNotification))      //Data frame arrives
6      {
7          ReceiveFrame();
8          if(corrupted(frame));
9              sleep();
10         if(seqNo == Rn)                  //Valid data frame
11         {
12             ExtractData();
13             DeliverData();                //Deliver data
14             Rn = Rn + 1;
15         }
16         SendFrame(Rn);                  //Send an ACK
17     }
18 }
```



Example 11.3

*Figure 11.11 shows an example of **Stop-and-Wait ARQ**. Frame 0 is sent and acknowledged. Frame 1 is lost and resent after the time-out. The resent frame 1 is acknowledged and the timer stops. Frame 0 is sent and acknowledged, but the acknowledgment is lost. The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged.*

Figure 11.11 *Flow diagram for Example 11.3*





Example 11.4

Assume that, in a Stop-and-Wait ARQ system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make a round trip. What is the bandwidth-delay product? If the system data frames are 1000 bits in length, what is the utilization percentage of the link?

Solution

The bandwidth-delay product is

$$(1 \times 10^6) \times (20 \times 10^{-3}) = 20,000 \text{ bits}$$



Example 11.4 (continued)

*The system can send 20,000 bits during the time it takes for the data to go from the sender to the receiver and then back again. However, the system sends only 1000 bits. We can say that the link utilization is only $1000/20,000$, or **5** percent. For this reason, for a link with a high bandwidth or long delay, the use of Stop-and-Wait ARQ wastes the capacity of the link.*



Example 11.5

What is the utilization percentage of the link in Example 11.4 if we have a protocol that can send up to 15 frames before stopping and worrying about the acknowledgments?

Solution

*The bandwidth-delay product is still 20,000 bits. The system can send up to 15 frames or 15,000 bits during a round trip. This means the utilization is $15,000/20,000$, or **75** percent. Of course, if there are damaged frames, the utilization percentage is much less because frames have to be resent.*



Stop-and-Wait ARQ

After each frame sent the host must wait for an ACK

❖ *inefficient use of bandwidth*

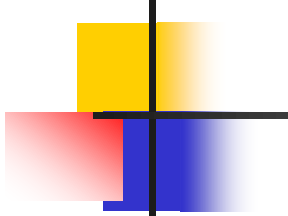
To improve efficiency ACK should be sent after multiple frames

Alternatives: Sliding Window protocol

- ✓ *Go-back-N ARQ*
- ✓ *Selective Repeat ARQ*

Pipelining: *A task is begun before the previous task has ended*

- ❖ ***There is no pipelining in stop and wait ARQ***
because we need to wait for a frame to reach the destination and be acknowledged before the next frame can be sent
- ❖ ***Pipelining improves the efficiency of the transmission***



Sliding window protocol

Sliding window protocols apply Pipelining :

- ✓ ***Go-Back-N ARQ***
- ✓ ***Selective Repeat ARQ***

- ***Sliding window protocols improve the efficiency multiple frames should be in transition while waiting for ACK. Let more than one frame to be outstanding.***
- ***Outstanding frames: frames sent but not acknowledged***
- ***We can send up to W frames and keep a copy of these frames(outstanding) until the ACKs arrive.***
- ***This procedures requires additional feature to be added :*sliding window****

Sliding window

Sequence Numbers

- *Sent frames are numbered sequentially*
- *Sequence number is stored in the header of the frame*
- *If the header of the frame allow m bits for the sequence number, the sequence numbers range from 0 to $(2^m - 1)$.*

The sequence numbers are modulo 2^m , where m is the size of the sequence number field in bits.

If $m = 3$, sequence number range from 0 to 7(8 numbers): 0, 1, 2, 3, 4, 5, 6, 7, 0, 1,

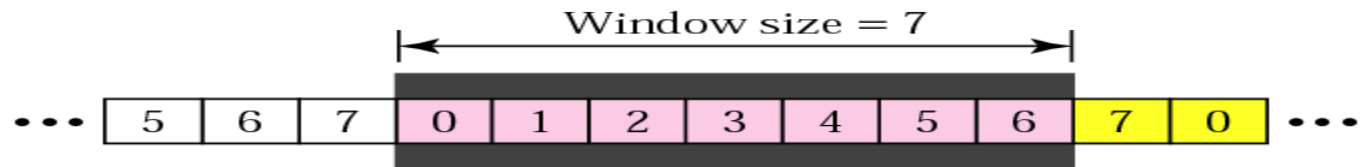
Sliding window

used to hold the unacknowledged outstanding frames (frames sent but not acknowledged)

Go_Back_N ARQ

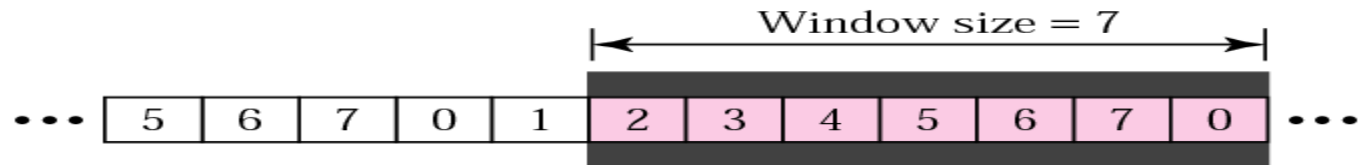
Sender sliding window

*The sender window is an abstract concept defining an imaginary box of size $2^m - 1$ (sequence numbers -1)
The sender window can slide one or more slots when a valid acknowledgment arrives.*



a. Before sliding

If $m = 3$; sequence numbers = 8 and
window size = 7



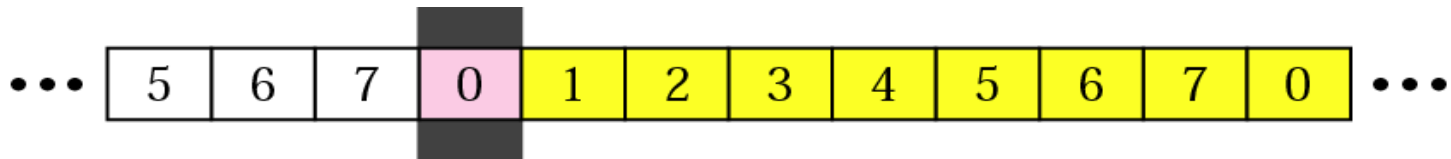
b. After sliding two frames

Acknowledged frames

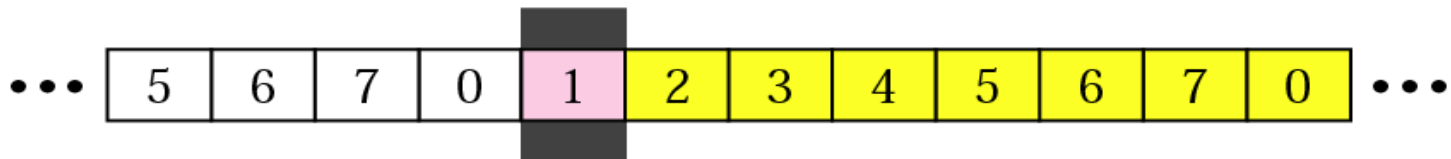
Go_Back_NARQ

Receiver sliding window

- *The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n .*
- *The window slides when a correct frame has arrived; sliding occurs one slot at a time.*



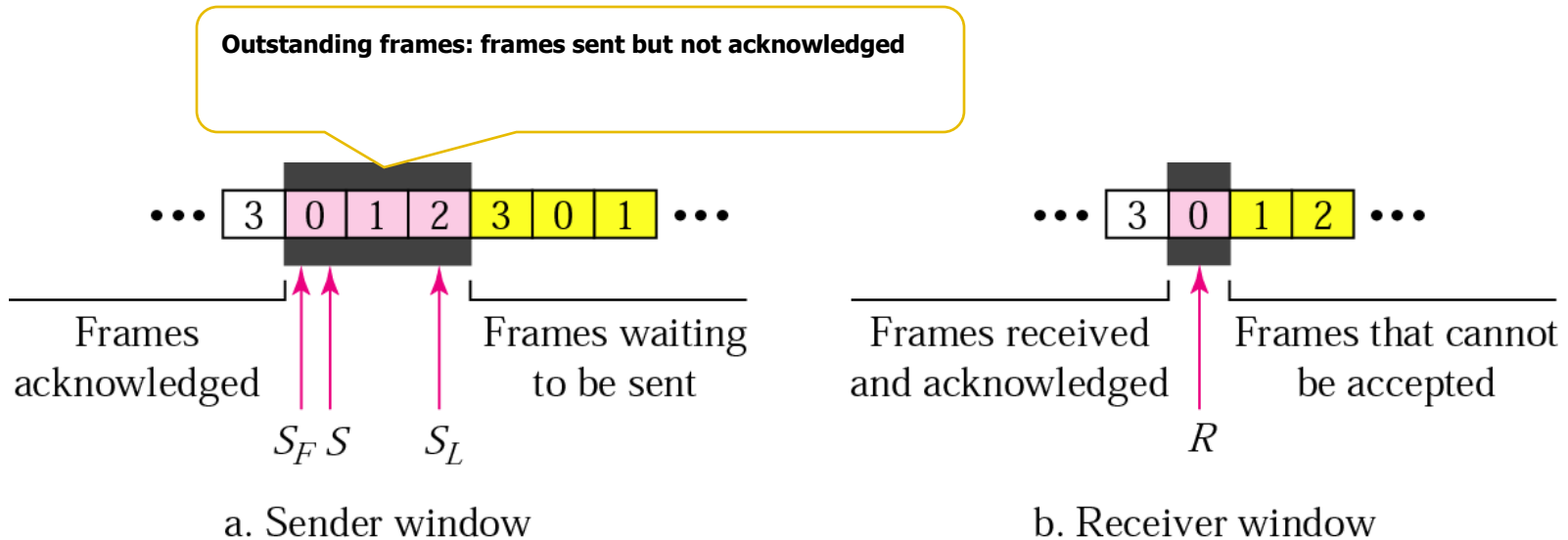
a. Before sliding



b. After sliding

Go-Back-N ARQ

control variables



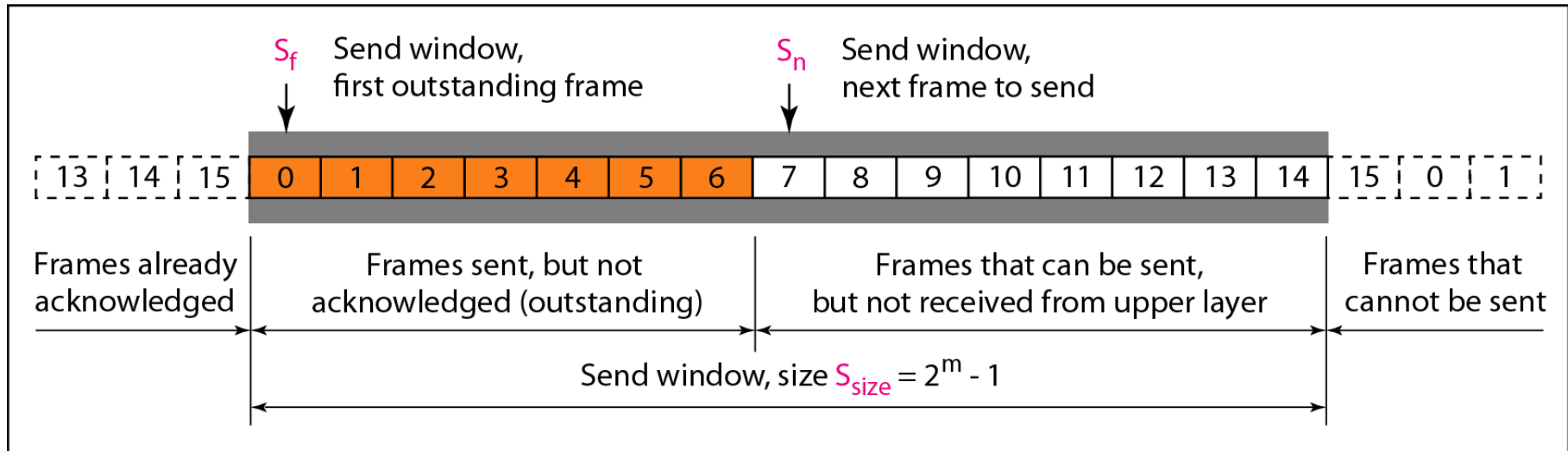
S: hold the sequence number of the recently sent frame

S_F: holds sequence number of the first frame in the window

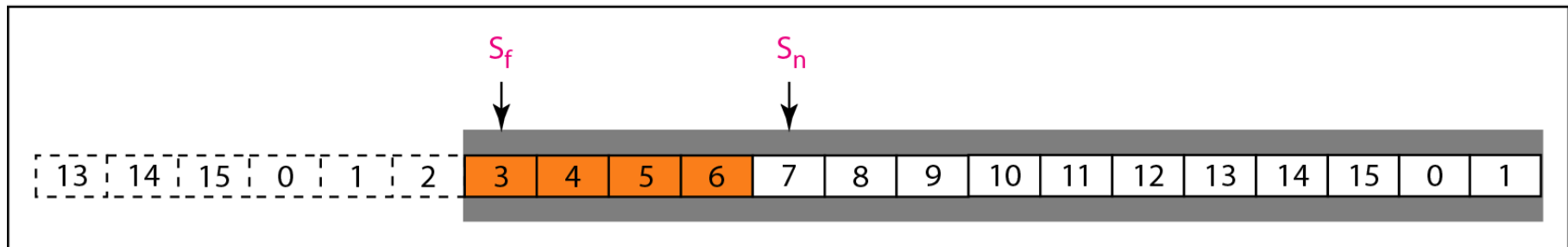
S_L: holds the sequence number of the last frame

R: sequence number of the frame expected to received

Go-Back-N ARQ



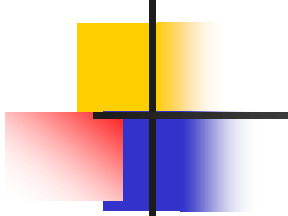
a. Send window before sliding



b. Send window after sliding

In Go-Back-N ARQ we use one timer for the first outstanding frame

- *The receiver sends a positive ACK if a frame has arrived safe and in order.*
- *if a frame is damaged or out of order ,the receiver is silent and will discard all subsequent frames*
- *When the timer of an unacknowledged frame at the sender site is expired , the sender goes back and resend all frames , beginning with the one with expired timer.(that is why the protocol is called Go-Back-N ARQ)*
- *The receiver doesn't have to acknowledge each frame received . It can send cumulative Ack for several frame*



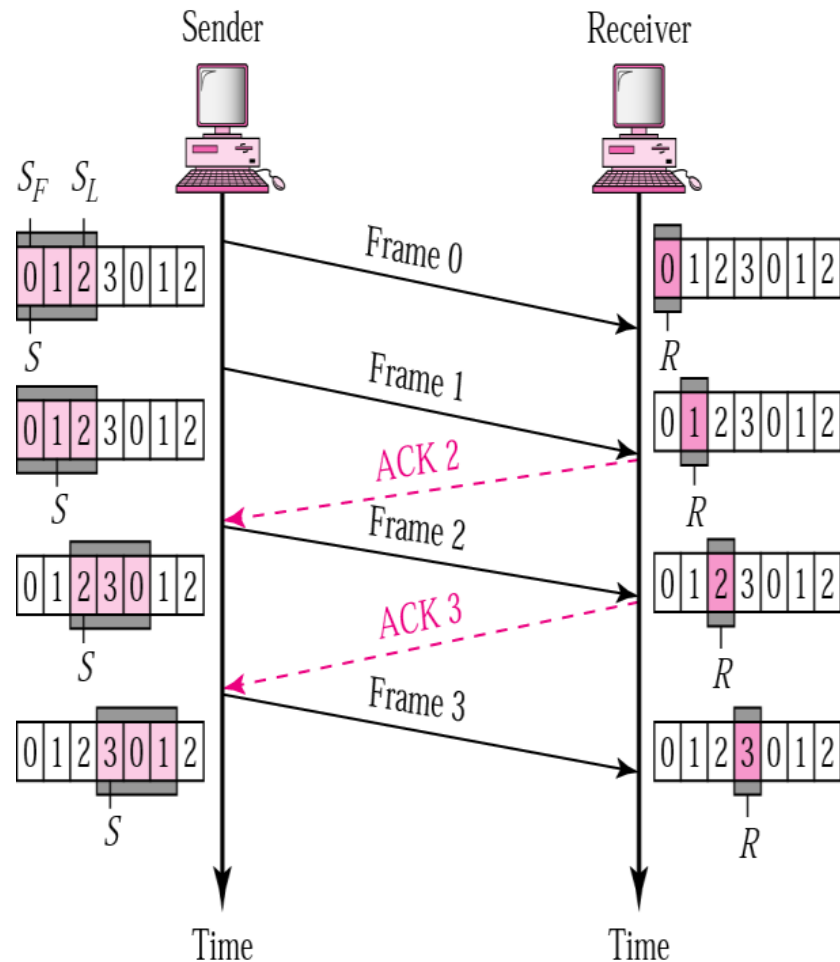
Go-Back-N ARQ

Example: The sender has sent frame 6 , and timer expires for frame 3(frame 3 has not been acknowledge); the sender goes back and resends frames 3, 4,5 and 6

Go-Back-N ARQ

Normal operation

- *How many frames can be transmitted without acknowledgment?*
- *ACK1 is not necessary if ACK2 is sent: Cumulative ACK*



Go-Back-N ARQ

Damage or Lost Frame

Correctly received out of order packets are not Buffered

What is the disadvantage of this?

